

Fundamentos de Python

Corpus de referencia para SOFIA.py — Sistema de Tutoría Socrática

Cubre: variables, condicionales, ciclos, funciones, listas, strings, diccionarios, recursión, clases, excepciones y archivos.

Capítulo 1: Variables y Tipos de Datos

Una variable es un nombre que referencia un valor almacenado en memoria. En Python no se declara el tipo — se infiere automáticamente según el valor asignado.

Tipos básicos

Los tipos fundamentales en Python son: int (enteros), float (decimales), str (cadenas de texto), bool (True/False) y NoneType (valor nulo).

```
x = 10          # int - número entero
precio = 3.14   # float - número decimal
nombre = "Ana"  # str - cadena de texto
activo = True   # bool - valor booleano
vacio = None    # NoneType - sin valor
```

```
# Verificar el tipo de una variable
print(type(x))      # <class 'int'>
print(type(nombre)) # <class 'str'>
```

Conversión de tipos

Python permite convertir entre tipos usando funciones integradas: int(), float(), str(), bool(). Esto se llama casting o conversión de tipo.

```
edad_texto = "25"
edad_numero = int(edad_texto)  # str → int
precio = float("3.99")         # str → float
texto = str(100)               # int → str

# Error común: sumar str con int sin convertir
# print("Edad: " + 25) # TypeError
print("Edad: " + str(25))      # correcto
```

Reglas de nombres de variables

Los nombres de variables deben comenzar con letra o guión bajo, pueden contener letras, números y guiones bajos, y son sensibles a mayúsculas. Por convención se usa snake_case.

```
nombre_completo = "Luis"  # correcto - snake_case
_privado = 42              # correcto - comienza con _
# 2variable = 10          # incorrecto - comienza con número
# mi-var = 5              # incorrecto - guión medio no permitido
```

Capítulo 2: Condicionales

Las estructuras condicionales permiten ejecutar código solo si se cumple una condición. Python usa if, elif y else con indentación obligatoria de 4 espacios.

Estructura básica if-elif-else

```
edad = 18

if edad >= 18:
    print("Mayor de edad")
elif edad >= 13:
    print("Adolescente")
else:
    print("Niño")

# Operadores de comparación: == != < > <= >=
# Operadores lógicos: and, or, not
nota = 75
if nota >= 90 and nota <= 100:
    print("Excelente")
elif nota >= 60 or nota == 55:
    print("Aprobado")
else:
    print("Reprobado")
```

Operador ternario

Python tiene una forma compacta de escribir un if-else en una sola línea llamada expresión condicional o operador ternario.

```
x = 10
resultado = "par" if x % 2 == 0 else "impar"
print(resultado) # par

# Equivalente a:
if x % 2 == 0:
    resultado = "par"
else:
    resultado = "impar"
```

Errores frecuentes en condicionales

El error más común es usar = (asignación) en lugar de == (comparación). Otro error es olvidar los dos puntos al final de if/elif/else.

```
# ERROR: asignación en lugar de comparación
# if x = 10: # SyntaxError

# CORRECTO:
if x == 10:
    print("x vale diez")

# ERROR: sin dos puntos
# if x > 0
```

```
#     print("positivo") # SyntaxError
```

```
# CORRECTO:
```

```
if x > 0:  
    print("positivo")
```

Capítulo 3: Ciclos e Iteración

Los ciclos permiten repetir un bloque de código. Python tiene dos tipos: for (para iterar sobre una secuencia) y while (mientras se cumpla una condición).

Ciclo for con range()

range(inicio, fin, paso) genera una secuencia de números. El valor final no se incluye. Es la forma más común de repetir código un número fijo de veces.

```
# Imprimir del 1 al 5
for i in range(1, 6):
    print(i)
# Salida: 1 2 3 4 5

# range con paso
for i in range(0, 10, 2):
    print(i)
# Salida: 0 2 4 6 8

# Iterar sobre una lista
frutas = ["manzana", "pera", "uva"]
for fruta in frutas:
    print(fruta)
```

Ciclo while

El ciclo while repite el bloque mientras la condición sea verdadera. Es fundamental tener una condición de parada para evitar ciclos infinitos.

```
contador = 0
while contador < 5:
    print(contador)
    contador += 1 # condición de parada: incrementar

# break termina el ciclo anticipadamente
# continue salta a la siguiente iteración
for i in range(10):
    if i == 5:
        break # para en 5
    if i % 2 == 0:
        continue # salta los pares
    print(i) # imprime: 1 3
```

Ciclo infinito — error común

Un ciclo while sin condición de parada corre indefinidamente. Siempre verifica que la variable de control cambie dentro del ciclo.

```
# ERROR: ciclo infinito
# x = 1
# while x > 0:
#     print(x) # x nunca cambia → loop infinito

# CORRECTO:
```

```
x = 1
while x > 0:
    print(x)
    x -= 1      # x decrece → el ciclo para
```

Capítulo 4: Funciones

Una función es un bloque de código reutilizable que realiza una tarea específica. Se define con `def` y puede recibir parámetros y retornar valores con `return`.

Definición y llamada

```
def saludar(nombre):
    """Retorna un saludo personalizado."""
    return "Hola, " + nombre

# Llamar la función
resultado = saludar("Luis")
print(resultado) # Hola, Luis

# Función con valor por defecto
def potencia(base, exponente=2):
    return base ** exponente

print(potencia(3))    # 9 (exponente=2 por defecto)
print(potencia(3, 3)) # 27
```

Return vs Print

`return` devuelve un valor que puede usarse en el programa. `print` solo muestra en pantalla pero no retorna nada (retorna `None`). Este es uno de los errores más comunes en principiantes.

```
def doble_print(x):
    print(x * 2)    # muestra pero no retorna

def doble_return(x):
    return x * 2    # retorna el valor

# Con print: no puedes usar el resultado
resultado = doble_print(5) # muestra 10
print(resultado)           # None ← no hay valor

# Con return: puedes usar el resultado
resultado = doble_return(5)
print(resultado)           # 10 ← valor real
print(resultado + 1)       # 11 ← se puede operar
```

Scope de variables

Las variables definidas dentro de una función son locales — no existen fuera de ella. Las variables globales existen en todo el programa pero modificarlas dentro de una función requiere la palabra clave `global`.

```
x = 10 # variable global

def mi_funcion():
    x = 20 # variable LOCAL (no modifica la global)
    print(x) # 20

mi_funcion()
print(x) # 10 ← la global no cambió
```

```
# Para modificar la global:
def modificar_global():
    global x
    x = 99

modificar_global()
print(x) # 99
```

Capítulo 5: Listas

Una lista es una colección ordenada y mutable de elementos. Puede contener elementos de diferentes tipos. Los índices comienzan en 0.

Operaciones básicas

```
numeros = [1, 2, 3, 4, 5]

# Acceso por índice (empieza en 0)
print(numeros[0]) # 1
print(numeros[-1]) # 5 (último elemento)

# Slicing [inicio:fin] (fin no se incluye)
print(numeros[1:3]) # [2, 3]
print(numeros[:3]) # [1, 2, 3]
print(numeros[2:]) # [3, 4, 5]

# Métodos principales
numeros.append(6) # agrega al final
numeros.insert(0, 0) # inserta en posición
numeros.pop() # elimina y retorna el último
numeros.remove(3) # elimina el valor 3
numeros.sort() # ordena en su lugar
largo = len(numeros) # longitud de la lista
```

Listas y funciones — mutabilidad

Las listas son mutables: si pasas una lista a una función, los cambios dentro de la función afectan la lista original. Esto es diferente a los tipos simples como int o str.

```
def agregar_elemento(lista, elemento):
    lista.append(elemento) # modifica la lista original

mi_lista = [1, 2, 3]
agregar_elemento(mi_lista, 4)
print(mi_lista) # [1, 2, 3, 4] ← cambió afuera

# Comprensión de listas
cuadrados = [x**2 for x in range(1, 6)]
print(cuadrados) # [1, 4, 9, 16, 25]

pares = [x for x in range(10) if x % 2 == 0]
print(pares) # [0, 2, 4, 6, 8]
```


Capítulo 6: Strings (Cadenas de Texto)

Un string es una secuencia inmutable de caracteres. Python tiene muchos métodos para manipular strings. Los strings se pueden indexar y hacer slicing igual que las listas.

Métodos principales

```
texto = "  Hola Mundo  "

print(texto.strip())      # "Hola Mundo" (quita espacios)
print(texto.lower())      # "  hola mundo  "
print(texto.upper())      # "  HOLA MUNDO  "
print(texto.replace("Hola", "Adiós")) # "  Adiós Mundo  "

palabras = "uno,dos,tres".split(",")
print(palabras)           # ['uno', 'dos', 'tres']

print(",".join(palabras)) # "uno,dos,tres"
print(len("Python"))      # 6
print("Py" in "Python")   # True
```

Formateo de strings

```
nombre = "Luis"
edad = 20

# f-strings (recomendado, Python 3.6+)
print(f"Me llamo {nombre} y tengo {edad} años")

# format()
print("Me llamo {} y tengo {} años".format(nombre, edad))

# Concatenación (menos eficiente)
print("Me llamo " + nombre + " y tengo " + str(edad) + " años")

# Strings multilinea
mensaje = """
Primera línea
Segunda línea
Tercera línea
"""
```

Capítulo 7: Diccionarios

Un diccionario es una colección de pares clave:valor. Las claves deben ser únicas e inmutables (strings, números, tuplas). Los valores pueden ser cualquier tipo.

Operaciones básicas

```
estudiante = {
    "nombre": "Luis",
    "edad": 20,
    "notas": [85, 90, 78]
}

# Acceso
print(estudiante["nombre"])          # Luis
print(estudiante.get("email", "N/A")) # N/A (valor por defecto)

# Modificar y agregar
estudiante["edad"] = 21
estudiante["email"] = "luis@uptc.edu.co"

# Eliminar
del estudiante["email"]
valor = estudiante.pop("edad") # elimina y retorna

# Iterar
for clave, valor in estudiante.items():
    print(f"{clave}: {valor}")

print(list(estudiante.keys())) # lista de claves
print(list(estudiante.values())) # lista de valores
```

Verificar existencia de claves

```
datos = {"a": 1, "b": 2}

# Forma correcta de verificar si existe una clave
if "a" in datos:
    print(datos["a"])

# Usar get() evita KeyError
print(datos.get("c"))          # None (no lanza error)
print(datos.get("c", 0))       # 0 (valor por defecto)

# ERROR común: acceder a clave inexistente
# print(datos["c"]) # KeyError: 'c'
```

Capítulo 8: Recursión

Una función recursiva es aquella que se llama a sí misma. Toda función recursiva necesita: un caso base (que detiene la recursión) y un caso recursivo (que reduce el problema).

Factorial y Fibonacci

```
# Factorial:  $n! = n * (n-1)!$ 
def factorial(n):
    if n == 0 or n == 1:    # caso base
        return 1
    return n * factorial(n - 1)    # caso recursivo

print(factorial(5))    # 120

# Fibonacci:  $fib(n) = fib(n-1) + fib(n-2)$ 
def fibonacci(n):
    if n <= 1:              # caso base
        return n
    return fibonacci(n-1) + fibonacci(n-2)    # recursivo

print(fibonacci(6))    # 8
```

Traza de recursión — cómo funciona

```
# factorial(3) se ejecuta así:
# factorial(3) → 3 * factorial(2)
#               factorial(2) → 2 * factorial(1)
#               factorial(1) → 1    (caso base)
#               factorial(2) → 2 * 1 = 2
# factorial(3) → 3 * 2 = 6

# Sin caso base → RecursionError (stack overflow)
def infinita(n):
    return infinita(n - 1)    # nunca para

# infinita(1) # RecursionError: maximum recursion depth exceeded
```

Capítulo 9: Clases y Objetos

Una clase es una plantilla para crear objetos. Un objeto es una instancia de una clase. La programación orientada a objetos (OOP) organiza el código en torno a objetos que tienen atributos (datos) y métodos (comportamiento).

Definición básica

```
class Estudiante:
    """Representa un estudiante universitario."""

    def __init__(self, nombre, codigo):
        """Constructor: se ejecuta al crear el objeto."""
        self.nombre = nombre    # atributo de instancia
        self.codigo = codigo
        self.notas = []

    def agregar_nota(self, nota):
        """Agrega una nota a la lista."""
        self.notas.append(nota)

    def promedio(self):
        """Calcula el promedio de notas."""
        if not self.notas:
            return 0
        return sum(self.notas) / len(self.notas)

    def __str__(self):
        """Representación legible del objeto."""
        return f"{self.nombre} ({self.codigo})"

# Crear instancias
e1 = Estudiante("Luis", "20231234")
e1.agregar_nota(85)
e1.agregar_nota(90)
print(e1.promedio()) # 87.5
print(e1)            # Luis (20231234)
```

Herencia

```
class Persona:
    def __init__(self, nombre):
        self.nombre = nombre

    def saludar(self):
        return f"Hola, soy {self.nombre}"

class Profesor(Persona):    # hereda de Persona
    def __init__(self, nombre, materia):
        super().__init__(nombre)    # llama al constructor padre
        self.materia = materia

    def presentar(self):
        return f"{self.saludar()}, dicto {self.materia}"

p = Profesor("Carlos", "Python")
```

```
print(p.presentar())  
# Hola, soy Carlos, dicto Python
```

Capítulo 10: Manejo de Excepciones

Las excepciones son errores que ocurren durante la ejecución. Python usa try/except para capturarlas y manejarlas sin que el programa se detenga abruptamente.

try / except / finally

```
# Estructura básica
try:
    numero = int(input("Ingresa un número: "))
    resultado = 10 / numero
    print(f"Resultado: {resultado}")
except ValueError:
    print("Error: debes ingresar un número válido")
except ZeroDivisionError:
    print("Error: no se puede dividir entre cero")
except Exception as e:
    print(f"Error inesperado: {e}")
finally:
    print("Este bloque siempre se ejecuta")
```

Excepciones más comunes

```
# IndexError: índice fuera de rango
lista = [1, 2, 3]
try:
    print(lista[99])
except IndexError as e:
    print(f"IndexError: {e}")

# KeyError: clave no existe en diccionario
d = {"a": 1}
try:
    print(d["b"])
except KeyError as e:
    print(f"KeyError: {e}")

# TypeError: operación con tipos incompatibles
try:
    print("texto" + 5)
except TypeError as e:
    print(f"TypeError: {e}")

# Lanzar excepción manualmente
def dividir(a, b):
    if b == 0:
        raise ValueError("El divisor no puede ser cero")
    return a / b
```

Capítulo 11: Manejo de Archivos

Python puede leer y escribir archivos de texto usando la función `open()`. Se recomienda usar `with open()` para garantizar que el archivo se cierre correctamente incluso si ocurre un error.

Leer y escribir archivos

```
# Escribir archivo
with open("datos.txt", "w", encoding="utf-8") as f:
    f.write("Primera línea\n")
    f.write("Segunda línea\n")

# Leer archivo completo
with open("datos.txt", "r", encoding="utf-8") as f:
    contenido = f.read()
    print(contenido)

# Leer línea por línea
with open("datos.txt", "r", encoding="utf-8") as f:
    for linea in f:
        print(linea.strip())

# Agregar contenido sin borrar (modo "a")
with open("datos.txt", "a", encoding="utf-8") as f:
    f.write("Tercera línea\n")
```

Modos de apertura

```
# Modos principales:
# "r" → leer (error si no existe)
# "w" → escribir (crea o sobrescribe)
# "a" → agregar al final
# "r+" → leer y escribir

import os

# Verificar si existe antes de leer
if os.path.exists("datos.txt"):
    with open("datos.txt", "r") as f:
        print(f.read())
else:
    print("El archivo no existe")

# Manejo seguro con excepciones
try:
    with open("archivo.txt", "r") as f:
        print(f.read())
except FileNotFoundError:
    print("Archivo no encontrado")
```

Resumen de Conceptos

Este documento cubre los 11 conceptos fundamentales de Python necesarios para el sistema SOFIA:

Concepto	Palabras clave
-----	-----
variables_y_tipos	int, float, str, bool, type(), asignación, valor
condicionales	if, elif, else, True, False, comparación, lógico
ciclos	for, while, range(), break, continue, iteración
funciones	def, return, parámetro, argumento, scope, docstring
listas	list, append, index, slice, sort, len, mutable
strings	str, upper, lower, split, join, format, f-string
diccionarios	dict, clave, valor, get(), items(), keys()
recursion	caso base, caso recursivo, factorial, fibonacci
clases	class, __init__, self, método, herencia, objeto
excepciones	try, except, raise, ValueError, TypeError, finally
archivos	open(), read(), write(), with, os.path, encoding